

Chapter 1

Background

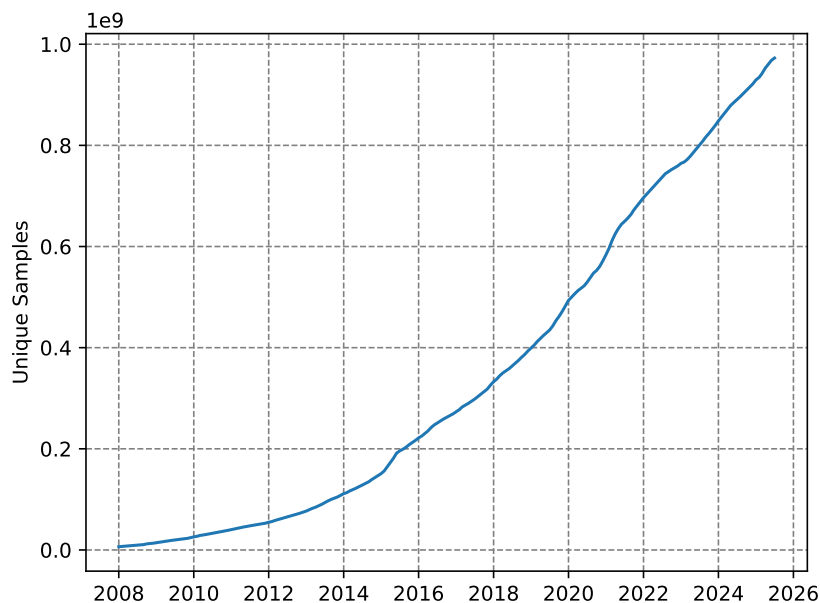


Figure 1.1: Observed growth in the number of unique forms of malware over time. Data taken from AV-TEST - The Independent IT-Security Institute, 2025

Malicious software, also known as malware, may be defined as software that has an adverse impact on the expected operation of a computer system. This encompasses the provision of access by an unauthorised user to an infected host, illegitimate extraction and modification of information, or denying access to the host by its users. They are considered one of the most pervasive threats to both individuals and organisations in our current digital threat landscape (European Union Agency for Cybersecurity, 2024; Alenezi et al., 2020).

The threat posed by malware has been seen in cyberattacks such as the 2022 Viasat hack (Quiquet, 2023), and the 2017 WannaCry incident (National Audit Office (NAO), 2017; Akbanov et al., 2019). In these cases, sophisticated malware were used to disrupt critical infrastructure including communications infrastructure across Europe, and critical systems used by the National Health Service.

In recent years, both the sophistication (Alenezi et al., 2020; Baezner et al., 2017) and frequency (Figure 1.1) of malware has increased significantly. As a result of this, and the potential damage caused by malicious software, there has been a great deal of interest into techniques for the reliable identification of malware (M. et al., 2023; Maniriho et al., 2024; Aboaoja et al., 2022). In this chapter, we provide a background on the characteristics of malware, and review techniques proposed by security researchers to better manage this threat.

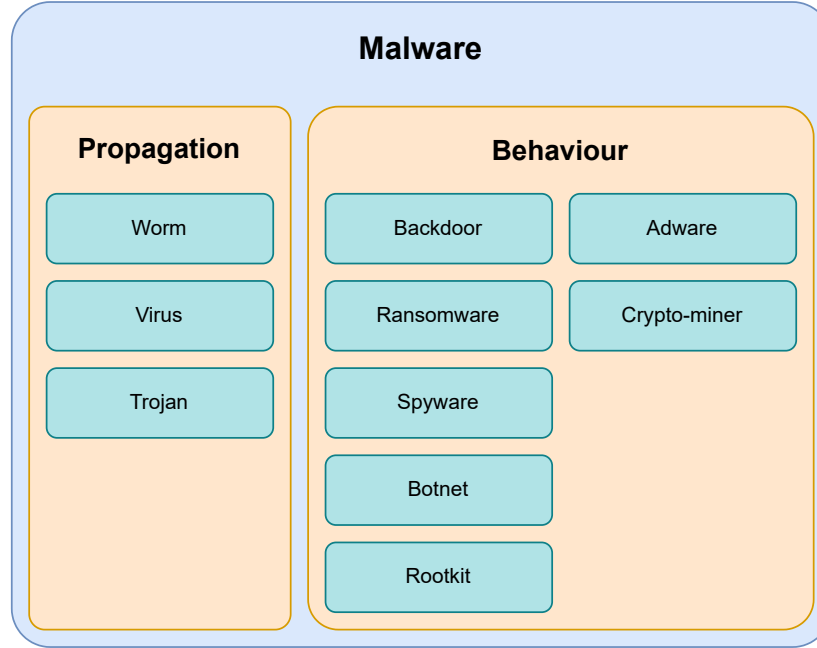


Figure 1.2: Categories of Malware, including both the means of propagation and behaviour on an infected host.

1.1 Malware

As outlined in the previous section, malware are a form of software that have a detrimental impact on the expected operation of an infected system. However, both this definition, and formal definitions, such as the one presented by (Kramer et al., 2010), that define malware as software that cause other programs to deviate from their intended behaviour in a harmful manner, fail to define what is meant by harmful. Thus, we define harmful behaviour as actions that are in violation of the Confidentiality-Integrity-Availability (CIA) triad (Cochran, 2024). This is a principle used to guide information security policies which outlines the need for a system to be protected against unauthorised access (Confidentiality), modification (integrity), and reliable (availability) to be considered secure.

Malware is often categorised based on its behaviour, and its method of propagation. We present the following taxonomy based on the work of Maniriho et al., 2024 shown in Figure 1.2 that outlines the different kinds of malware seen by security researchers, and how they are propagate. Note that categories within this taxonomy are not mutually exclusive, as they often belong to multiple categories at once.

- **Worm** — Malicious software that automatically propagates throughout a network.
- **Virus** — Programs that modify other programs to replicate themselves throughout an infected host.
- **Trojans** — Malware that disguises itself as a legitimate program to infect a host.
- **Backdoor** — Malware that provides covert remote access to infected systems.
- **Ransomware** — Malware that denies access to systems or information until a fee is paid.
- **Spyware** — Malware that spy on the user without their knowledge, often extracting information such as credentials, keyboard logs, and live screenshots of the user’s activity.
- **Botnet** — Malware that integrate infected systems into a coordinated network of agents.
- **Rootkit** — Malware providing privileged access to an infected system.
- **Adware** — Malware that display advertisements on the host machine.
- **Crypto-miner** — Malware that hijack the user’s hardware to mine cryptocurrencies.

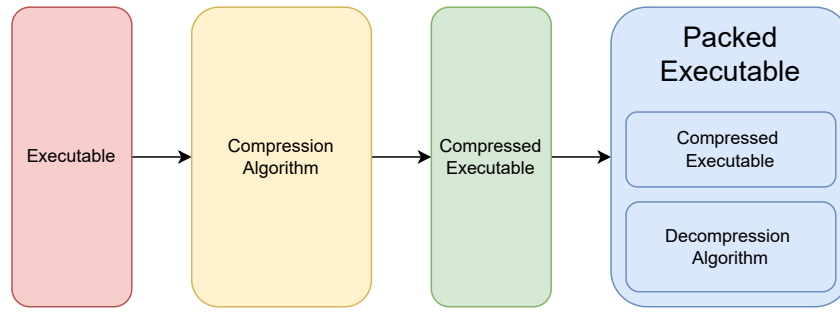


Figure 1.3: How binary packing can be used to reduce the size of an executable program, and obscure its behaviour.

1.1.1 Evasion Techniques

The earliest forms of malware detection used a combination of unique signatures associated with malicious programs, and a set of heuristics to identify malicious programs (A. Saeed et al., 2013). In response to this, the developers of malicious software began using evasion techniques that aim to avoid detection by these systems. This was done in order to maximise the amount of potential damage the software is able to do to an infected host before it is identified and removed. These evasion techniques include obfuscation, environmental awareness, and living-off-the-land. In the following section, we discuss these techniques, and why they are effective at defeating existing malware detection.

Obfuscation

Obfuscation is defined as the act of making something more challenging to understand. In the context of malware, it relates to techniques that alter the syntactic structure of a program, while maintaining its original behavioural characteristics.

Register re-assignment is the process of randomly reallocating registers used by the program (Balakrishnan et al., 2005) as it executes. These are small areas of memory within the CPU that store information used during the program’s execution. By changing which registers are used to store information, the physical structure of the program is changes, yet its behaviour remains the same. This defeats simple malware detection solutions that use the unique cryptographic hashes of a program as its signature.

Dead code insertion is another simple form of obfuscation that inserts instructions which have no impact on the state of the program’s execution. These instructions include the no-operation (`nop`), swapping the value stored in a register with itself, and adding zero to a register to name a few. This again modifies the physical structure of a program, without impacting its behaviour. This is not limited to singular instructions, and can include the insertion of functions which are never executed which can make the program more challenging to identify. However, dead code is relatively easy to identify and remove.

Code transposition is another form of obfuscation that modifies the order of code the program (Christodorescu et al., 2004). To ensure identical behaviour characteristics, small units of code, known as basic blocks that do not contain jumps, such as loops or conditional statements, are moved to another location within the binary. To ensure the program behaves as expected, all references to the relocated basic blocks are updated with its new location. This evades detection in a similar way to the obfuscation techniques discussed previously, by creating unique representations of the same program that give produce different signatures not recognised by the analysis software.

While these techniques are able to avoid detection by early approaches to detecting malware, more sophisticated systems are able to recognise them as malicious even though their representation may be different. To achieve this, an intermediate representation of the program may be used, that normalise the program structure so that an obfuscated appears identical to the original. To bypass detection such approaches, more sophisticated forms of obfuscation are needed. One such example of this is known as binary packing. To produce a packed binary, the malicious code is first compressed and optionally encrypted, where it is then written as data to another executable that contains the logic to decode and execute the original code (Figure 1.3). Without running the packed binary, it is impossible to retrieve the encoded code, which bypasses detection by more sophisticated solutions.

However, the portion of the packed binary that decodes and executes the encoded payload typically remains constant between representations, and can be used to identify potentially malicious packed programs. Some mal-

ware developers employ the aforementioned obfuscation techniques to produce unique decoders between different representations which is known as Oligomorphic and Polymorphic malware (You et al., 2010).

Environmental Awareness

As a result of the previously discussed obfuscation techniques, relying on the static structure of a program may be insufficient to reliably detect malware. It is for this reason that dynamic features are considered to be more robust indicators of malicious programs, as the behaviour of a program is invariant to obfuscation. These dynamic features are observed at execution time and can include sequences of system API calls, modifications to the filesystem and system configuration, and internet traffic. However, this requires running a potentially malicious program, which could have harmful effects on the host system. It is for this reason that execution of a potentially malicious program occurs within a sandbox, an environment where the harmful actions can occur with no implications on the security of the system. This sandbox is typically a virtual machine or a containerized application, that’s representative of the software’s intended target.

To obscure the behaviour of the program, the developers of these malicious programs will attempt to check if the program is running within a virtual machine. Common heuristics used by malicious programs include system hardware characteristics (CPU core count, RAM size, etc), the presence of shared libraries used to communicate between the sandbox and its host, and the number of processes running on the potential host. If the program determines that it is likely to be running within a sandbox, it will refuse to execute the malicious portion of the code, and exit early. This prevents the observer from gaining any information about its behavioural characteristics, preventing the creation of dynamic malicious signatures.

Living-off-the-land

In recent years, the sophistication of malware has increased dramatically. Living off the land refers to an evasion technique used by advanced malware that significantly reduces the ability of existing security solutions to detect the presence of malware on a host system.

Upon gaining access to a system, often through software vulnerabilities, or social engineering, the malicious software will make use of trusted programs already present to further compromise the host. For example, certain tools in windows machines can make changes to the system registry, which is a trusted location that isn’t typically analysed by anti-malware software. It remains consistent between system restarts, and includes a list of programs that should always be executed upon turning on. This allows a threat actor to install malicious software within a trusted location, that is persistent between restarts, that is not saved to the host’s disk. This is known as fileless malware.

As the malware does not exist on disk, it is extremely difficult to detect by conventional means. Furthermore, as they rely on the functionality of trusted system programs, which are highly likely to be present on all targets.

1.2 Related Work

Malware detection is a challenging problem, and many approaches have been proposed since the first forms of malicious software were seen in the 1980s. The earliest form of malware detection used a database containing the unique signatures of malicious programs. While similar signature based approaches are still in use today, there is much interest in advanced methods, that are robust to both obfuscation and other forms of evasion, and able to identify novel forms of malware (M. et al., 2023; Merabet et al., 2019; Aboaoja et al., 2022; Idika, Mathur, 2007). These advanced techniques employ a wide variety of technologies, such as multiple sequence alignment of API call sequences (Ki et al., 2015), machine learning (Kamboj et al., 2023), and deep learning.

- **Static** — Features extracted from the structure of a program
- **Dynamic** — Features extracted during the execution of the program

Static approaches use features extracted from the contents of an executable, and do not require executing the program in question. Common features used to identify malware include greyscale images of the executable (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023), printable strings (Tian et al., 2009; Singh et al., 2020; Schultz et al., 2001), and sequences of instructions (Kakisim et al., 2022; Wang et al., 2021; Darem et al., 2021).

In Schultz et al., 2001, the authors use static features including printable strings to train various machine learning classifiers for malware detection. In their experiments, they found that the Naive Bayes algorithm was best suited

to the problem, achieving 97% classification accuracy. This is one of the first papers that introduce the usage of machine learning in this context, and the high accuracy of their experiment shows its potential over conventional signature or heuristic methods. However, the dataset used in this work was small, and featured an unbalanced class distribution of 3265 malicious programs, and only 1001 benign which would likely have an adverse effect on the generalisability of their proposed models. Furthermore, malware often include obfuscated printable strings, that are decoded during execution, which would be sufficient to defeat their approach.

Much of the literature relating to static malware detection use convolutional neural networks to distinguish between malicious and benign programs (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023). In Aslan et al., 2021, the authors represent an executable as a greyscale image, and train a convolutional neural network (CNN) to detect malware. While their proposed approach is able to effectively distinguish between executable classes, CNNs require constant input size, and in datasets where there is high variation in size between classes, compression artifacts may cause the model to struggle with classification. In Sharma et al., 2019, the authors also use a CNN to detect malware. Instead of using a 2-dimensional CNN however, the authors opt to use a 1-dimensional variant. This led to better performance than other CNN based approaches, as the vertical spatial dependence, which is largely irrelevant in malware detection, is removed. However, it still has the same requirement as the previously discussed approach.

Instruction sequences have also been explored as static features in malware detection. For example, in (Attaluri et al., 2009; Runwal et al., 2012), the authors utilise hidden markov models and achieve promising results. As these models consider what actions are being performed by the host system without having to execute the program, they offer a solution that we believe to be more resilient to obfuscation than other static approaches.

Extending upon the idea of utilising instruction sequences for malware detection, (Demirci et al., 2022) outline an approach in which they propose the use of stacked bidirectional Long-Short Term Memory (BiLSTM) and pre-trained language models including BERT (Devlin et al., 2019), and GPT-2 (Radford et al., n.d.). In their investigation, the BiLSTM model was able to achieve 98% classification accuracy whereas the pre-trained transformer models achieved only 77%. Transformer models have been shown to be a powerful technique for sequence classification problems, and as such, this result is surprising. We believe this to be a result of the corpora used to pre-train the transformer models used in their investigation, which are comprised of mostly english texts. The understanding of these models would thus be limited to languages similar to english, and may struggle to understand the intricacies of assembly language, impacting their ability to reliably classify malware.

Previous research has also established the use of pre-trained language models in malware detection on the metadata of an executable. In Rahali et al., 2021, the authors fine-tune BERT on text gathered from the manifest file found in android applications. This file contains information pertaining to the activities, background tasks, event listeners, and permissions granted to an application. Their model was able to achieve 97% binary classification accuracy, in comparison to (Demirci et al., 2022) which was only able to achieve 77% with the same model, trained on instruction sequences. We believe this discrepancy to be a result of the language found in these manifest files being more closely related to the pre-training corpora of the BERT model. This would enable their model to better understand the information found in the manifest, and its use in distinguishing between benign and malicious executables. Their approach however, is limited to environments where a file containing metadata are required, such as mobile operating systems, making it unsuitable for desktop malware detection.

The performance of static malware detection approaches vary, depending on the detection method used, the availability of data, and other factors, but it is well established that they are particularly vulnerable to obfuscation. In Bacci et al., 2018, the authors investigate the impact of obfuscation on the performance of static and dynamic malware detectors on android devices. They found that while dynamic methods are not significantly impacted by the presence of obfuscation, static techniques are more affected. However, by training static methods on obfuscated programs, it is possible to mitigate this problem to some extent.

Furthermore, dynamic methods generally tend to be more capable than static methods (Damodaran et al., 2017). This is somewhat to be expected, as behaviour is more indicative of malicious actions, especially when obfuscation is used to hide a program’s intent. However, dynamic methods require a sandbox to execute a program for analysis, without causing damage to a host. This typically takes the form of a remote, virtual machine as running such a sandbox locally would have a significant computational cost. As a result of this, dynamic methods require both significant compute resources, and an internet connection, which is not guaranteed. It is for this reason that static methods, that can identify potentially malicious programs, without an internet connection is needed for enhanced protection from this threat.

In addition to the high resource requirements associated with dynamic feature extraction, sophisticated evasion techniques such as environmental awareness and living off the land provide malware developers with the capabilities to obscure the dynamic features. For example, by either preventing execution within virtual machines, or using

trusted binaries already located on a host system. Furthermore, novel obfuscation techniques, such as the one proposed by Li et al., 2023, is able to obscure which system API calls are made by a program. This is a widely used dynamic feature for detecting malware, and the proposed technique enables the malware to call these system APIs without triggering system hooks used to track which methods are called during a program’s execution. Therefore, despite offering better performance than static methods, dynamic methods are also vulnerable to evasion techniques.

1.3 Future Research Directions

Our analysis of the literature relating to this field, we have found that there are a number of promising areas of investigation. Static approaches, while less robust to structural obfuscation, are less resource intensive and are able to be run locally. Dynamic features offer better performance than its static counterpart, yet are still vulnerable to advanced evasion techniques that may render them less effective than static. Furthermore, the requirements associated with dynamic feature extraction are a significant barrier to their deployment, and demonstrate the need for efficient and effective static methods for detecting malware.

We propose investigating a transformer model, that firstly learns the semantics of assembly language, then fine-tuned on a downstream sequence classification problem. While transformers can be expensive to run locally, we believe that the simple nature of the language will allow us to create a less complex model that is able to reliably and accurately identify malware.

Bibliography

- A. Saeed, Imtithal, Ali Selamat, and Ali M. A. Abuagoub (Apr. 18, 2013). “A Survey on Malware and Malware Detection Systems”. In: *International Journal of Computer Applications* 67.16, pp. 25–31. ISSN: 09758887. DOI: 10.5120/11480-7108. URL: <http://research.ijcaonline.org/volume67/number16/pxc3887108.pdf> (visited on 07/29/2025).
- Aboaoja, Faitouri A. et al. (Jan. 2022). “Malware Detection Issues, Challenges, and Future Directions: A Survey”. In: *Applied Sciences* 12.17 (17), p. 8482. ISSN: 2076-3417. DOI: 10.3390/app12178482. URL: <https://www.mdpi.com/2076-3417/12/17/8482> (visited on 12/10/2024).
- Akbanov, Maxat, Vassilios G Vassilakis, and Michael D Logothetis (2019). “WannaCry Ransomware: Analysis of Infection, Persistence, Recovery Prevention and Propagation Mechanisms”. In: *Journal of Telecommunications and Information Technology* 1, pp. 113–124.
- Alenezi, Mohammed et al. (Dec. 1, 2020). “Evolution of Malware Threats and Techniques: A Review”. In: *International Journal of Communication Networks and Information Security* 12, p. 326. DOI: 10.17762/ijcnis.v12i3.4723.
- Aslan, Ömer and Abdullah Asim Yilmaz (2021). “A New Malware Classification Framework Based on Deep Learning Algorithms”. In: *IEEE Access* 9, pp. 87936–87951. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3089586. URL: <https://ieeexplore.ieee.org/document/9455368> (visited on 07/28/2025).
- Attaluri, Srilatha, Scott McGhee, and Mark Stamp (May 1, 2009). “Profile Hidden Markov Models and Metamorphic Virus Detection”. In: *Journal in Computer Virology* 5.2, pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. URL: <https://doi.org/10.1007/s11416-008-0105-1> (visited on 02/25/2025).
- AV-TEST - The Independent IT-Security Institute (2025). *AV-ATLAS - Malware & PUA*. AV-ATLAS - Malware & PUA. URL: <https://portal.av-atlas.org/malware> (visited on 12/19/2024).
- Bacci, Alessandro et al. (2018). “Impact of Code Obfuscation on Android Malware Detection Based on Static and Dynamic Analysis”. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. 4th International Conference on Information Systems Security and Privacy. Funchal, Madeira, Portugal: SCITEPRESS - Science and Technology Publications. DOI: 10.5220/0006642503790385. URL: <http://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0006642503790385> (visited on 07/21/2025).
- Baezner, Marie and Patrice Robin (Oct. 18, 2017). *Stuxnet*. Report 4. ETH Zurich. DOI: 10.3929/ethz-b-000200661. URL: <https://www.research-collection.ethz.ch/handle/20.500.11850/200661> (visited on 07/22/2025).
- Balakrishnan, Arini and Chloe Schulze (Dec. 19, 2005). “Code Obfuscation Literature Survey”. In: Christodorescu, Mihai and Somesh Jha (Mar. 1, 2004). “Static Analysis of Executables to Detect Malicious Patterns”. In: 12.
- Cochran, Kodi A. (2024). “The CIA Triad: Safeguarding Data in the Digital Realm”. In: *Cybersecurity Essentials: Practical Tools for Today’s Digital Defenders*. Ed. by Kodi A. Cochran. Berkeley, CA: Apress, pp. 17–32. ISBN: 979-8-8688-0432-8. DOI: 10.1007/979-8-8688-0432-8_2. URL: https://doi.org/10.1007/979-8-8688-0432-8_2 (visited on 07/22/2025).
- Damodaran, Anusha et al. (Feb. 1, 2017). “A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection”. In: *Journal of Computer Virology and Hacking Techniques* 13.1, pp. 1–12. ISSN: 2263-8733. DOI: 10.1007/s11416-015-0261-z. URL: <https://doi.org/10.1007/s11416-015-0261-z> (visited on 07/29/2025).
- Darem, Abdulbasit et al. (Dec. 1, 2021). “Visualization and Deep-Learning-Based Malware Variant Detection Using OpCode-level Features”. In: *Future Generation Computer Systems* 125, pp. 314–323. ISSN: 0167-739X. DOI: 10.1016/j.future.2021.06.032. URL: <https://www.sciencedirect.com/science/article/pii/S0167739X21002272> (visited on 07/28/2025).

- Demirci, Deniz et al. (2022). “Static Malware Detection Using Stacked BiLSTM and GPT-2”. In: *IEEE Access* 10, pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. URL: https://ieeexplore.ieee.org/document/9785789?utm_source=chatgpt.com (visited on 02/25/2025).
- Devlin, Jacob et al. (May 24, 2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. URL: <http://arxiv.org/abs/1810.04805> (visited on 02/28/2025). Pre-published.
- European Union Agency for Cybersecurity (2024). *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office. URL: <https://data.europa.eu/doi/10.2824/0710888> (visited on 02/21/2025).
- Habibi, Omar, Mohammed Chemmakha, and Mohamed Lazaar (Aug. 1, 2023). “Performance Evaluation of CNN and Pre-trained Models for Malware Classification”. In: *Arabian Journal for Science and Engineering* 48.8, pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. URL: <https://doi.org/10.1007/s13369-023-07608-z> (visited on 02/25/2025).
- Idika, Mathur (Feb. 1, 2007). “A Survey of Malware Detection Techniques”. In: .
- Jain, M., W. Andreopoulos, and M. Stamp (2020). “Convolutional Neural Networks and Extreme Learning Machines for Malware Classification”. In: *Journal of Computer Virology and Hacking Techniques* 16.3, pp. 229–244. DOI: 10.1007/s11416-020-00354-y.
- Kakisim, Arzu Gorgulu, Sibel Gulmez, and Ibrahim Sogukpinar (Mar. 1, 2022). “Sequential Opcode Embedding-Based Malware Detection Method”. In: *Computers & Electrical Engineering* 98, p. 107703. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2022.107703. URL: <https://www.sciencedirect.com/science/article/pii/S0045790622000210> (visited on 07/28/2025).
- Kamboj, Akshit et al. (Mar. 1, 2023). “Detection of Malware in Downloaded Files Using Various Machine Learning Models”. In: *Egyptian Informatics Journal* 24.1, pp. 81–94. ISSN: 1110-8665. DOI: 10.1016/j.eij.2022.12.002. URL: <https://www.sciencedirect.com/science/article/pii/S111086652200072X> (visited on 07/28/2025).
- Ki, Youngjoon, Eunjin Kim, and Huy Kang Kim (June 1, 2015). “A Novel Approach to Detect Malware Based on API Call Sequence Analysis”. In: *International Journal of Distributed Sensor Networks* 11.6, p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. URL: <https://doi.org/10.1155/2015/659101> (visited on 11/02/2024).
- Kramer, Simon and Julian C. Bradfield (May 1, 2010). “A General Definition of Malware”. In: *Journal in Computer Virology* 6.2, pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. URL: <https://doi.org/10.1007/s11416-009-0137-1> (visited on 02/21/2025).
- Li, Yang et al. (Jan. 2023). “APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity”. In: *Applied Sciences* 13.16 (16), p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. URL: <https://www.mdpi.com/2076-3417/13/16/9056> (visited on 02/25/2025).
- M., Gopinath and Sibi Chakkaravarthy Sethuraman (Feb. 1, 2023). “A Comprehensive Survey on Deep Learning Based Malware Detection Techniques”. In: *Computer Science Review* 47, p. 100529. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2022.100529. URL: <https://www.sciencedirect.com/science/article/pii/S1574013722000636> (visited on 11/20/2024).
- Maniriho, Pascal, Abdun Naser Mahmood, and Mohammad Javed Morshed Chowdhury (Mar. 1, 2024). “A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions”. In: *Journal of Systems and Software* 209, p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. URL: <https://www.sciencedirect.com/science/article/pii/S0164121223003163> (visited on 02/18/2025).
- Merabet, Hoda El and Abderrahmane Hajraoui (2019). “A Survey of Malware Detection Techniques Based on Machine Learning”. In: *International Journal of Advanced Computer Science and Applications* 10.1. ISSN: 21565570, 2158107X. DOI: 10.14569/IJACSA.2019.0100148. URL: <http://thesai.org/Publications/ViewPaper?Volume=10&Issue=1&Code=ijacsa&SerialNo=48> (visited on 11/20/2024).
- National Audit Office (NAO) (Oct. 2017). *Investigation: WannaCry Cyber Attack and the NHS*. London, England: National Audit Office. URL: <https://web.archive.org/web/20241212140331/https://www.nao.org.uk/reports/investigation-wannacry-cyber-attack-and-the-nhs/>.
- Ni, Sang, Quan Qian, and Rui Zhang (Aug. 1, 2018). “Malware Identification Using Visualization Images and Deep Learning”. In: *Computers & Security* 77, pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/j.cose.2018.04.005. URL: <https://www.sciencedirect.com/science/article/pii/S0167404818303481> (visited on 02/21/2025).
- Quiquet, François (Oct. 10, 2023). *An analysis of the Viasat cyber attack with the MITRE ATT&CK® framework*. Space & Cybersecurity Info. URL: <https://www.spacesecurity.info/an-analysis-of-the-viasat-cyber-attack-with-the-mitre-attck-framework/> (visited on 07/21/2025).
- Radford, Alec et al. (n.d.). “Language Models Are Unsupervised Multitask Learners”. In: ().

- Rahali, Abir and Moulay A. Akhloufi (Oct. 2021). “MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers”. In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. URL: <https://ieeexplore.ieee.org/abstract/document/9659287> (visited on 02/25/2025).
- Runwal, Neha, Richard M. Low, and Mark Stamp (May 1, 2012). “Opcode Graph Similarity and Metamorphic Detection”. In: *Journal in Computer Virology* 8.1, pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. URL: <https://doi.org/10.1007/s11416-012-0160-5> (visited on 02/25/2025).
- Schultz, M.G. et al. (May 2001). “Data Mining Methods for Detection of New Malicious Executables”. In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*. Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. URL: <https://ieeexplore.ieee.org/document/924286> (visited on 02/21/2025).
- Sharma, Arindam, Pasquale Malacaria, and given-i=MHR family=Khouzani given=MHR (June 2019). “Malware Detection Using 1-Dimensional Convolutional Neural Networks”. In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pp. 247–256. DOI: 10.1109/EuroSPW.2019.00034. URL: <https://ieeexplore.ieee.org/document/8802378> (visited on 07/28/2025).
- Singh, Jagsir and Jaswinder Singh (May 1, 2020). “Detection of Malicious Software by Analyzing the Behavioral Artifacts Using Machine Learning Algorithms”. In: *Information and Software Technology* 121, p. 106273. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106273. URL: <https://www.sciencedirect.com/science/article/pii/S0950584920300239> (visited on 07/28/2025).
- Tian, Ronghua et al. (2009). “An Automated Classification System Based on the Strings of Trojan and Virus Families”. In: *2009 4th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 23–30. DOI: 10.1109/MALWARE.2009.5403021.
- Wang, Haojun et al. (2021). “Deep Learning and Regularization Algorithms for Malicious Code Classification”. In: *IEEE Access* 9, pp. 91512–91523. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3090464. URL: <https://ieeexplore.ieee.org/document/9459716> (visited on 07/28/2025).
- You, Ilsun and Kangbin Yim (2010). “Malware Obfuscation Techniques: A Brief Survey”. In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.